

Pack 28 (Desktop + Shared) — Desktop-Side Receipt Signature Verification + Public-Key Pinning (real code)

RemoteDesk · Pack 28 (Desktop + Shared) — Desktop-Side Receipt Signature Verification + Public-Key Pinning (real code)

Codex / Claude Code handoff. Direct continuation of Packs 22 → 27. The server now signs receipts (26) with KMS-protected, rotatable keys (27). This slice closes the loop on the DESKTOP: after a session ends and the server returns the signed receipt, the host independently verifies the Ed25519 signature offline AND pins the org's public key on first use, so a swapped/forged signing key is detected on the client, not just trusted from the server. Build order: shared webcrypto-ed25519 verify adapter + pinning contract → desktop key-pin store → desktop receipt verifier → UI badge → wiring → tests.

Scope: let the host cryptographically confirm its own receipts without trusting the server's word, and detect signing-key substitution via trust-on-first-use pinning. **Verification is offline + content-free. Pin mismatch fails closed (shows unverified, never silently accepts). No private keys on the desktop. Reuses Pack 26 shared verifier + Pack 22 chain verifier.**

Tag legend: **[NEW]** = create file. **[MODIFY]** = patch existing file by hand (merge, don't overwrite).

Files created / modified / not touched

CREATED [NEW]	
packages/shared/src/permissions/webcryptoEd25519.ts	
packages/shared/src/permissions/keyPinning.ts	
apps/desktop/src/renderer/src/services/signingKeyPinStore.ts	
apps/desktop/src/renderer/src/services/receiptVerifyClient.ts	
apps/desktop/src/renderer/src/components/ReceiptVerificationBadge.tsx	
tests/permissions/webcryptoEd25519.test.ts	
tests/permissions/keyPinning.test.ts	
tests/permissions/receiptVerifyClient.test.ts	
MODIFIED [MODIFY]	
packages/shared/src/permissions/index.ts	(export webcryptoEd25519 + keyPinning)
apps/desktop/src/renderer/src/services/consentReceiptService.ts	(verify signature after issue)
apps/desktop/src/renderer/src/components/SessionSummaryDialog.tsx	(show verification badge)
NOT TOUCHED (intentionally)	
apps/desktop/src/main/index.ts , preload/index.ts	(no native; no bridge)
apps/api/**	(signing/publishing already exist in 26/27)
packages/shared/src/permissions/receiptSignature.ts	(Pack 26 verify contract reused as-is)

Part A — Shared: WebCrypto Ed25519 verify + pinning contract

FILE: packages/shared/src/permissions/webcryptoEd25519.ts

```
import type { Ed25519Verify } from './receiptSignature';

/**
 * Ed25519 signature verification using the renderer's WebCrypto (SubtleCrypto).
 * This is the browser/Electron-renderer counterpart to the API's node:crypto
 * verifier — SAME algorithm, so a signature made server-side verifies here.
 * Provides the injectable Ed25519Verify the shared verifyReceiptSignature needs.
 *
 * Keys + signatures are base64url (matching the server). No private material.
 */
export const webcryptoEd25519Verify: Ed25519Verify = async (
  publicKeyB64Url: string,
  message: string,
  signatureB64Url: string,
): Promise<boolean> => {
  try {
    const key = await crypto.subtle.importKey(
      'raw',
      fromB64url(publicKeyB64Url),
      { name: 'Ed25519' },
      false,
      ['verify'],
    );
    return await crypto.subtle.verify(
      { name: 'Ed25519' },
      key,
      fromB64url(signatureB64Url),
      new TextEncoder().encode(message),
    );
  } catch {
    return false; // unsupported/invalid => fail closed
  }
};

function fromB64url(s: string): Uint8Array {
  const b64 = s.replace(/-/g, '+').replace(/_/g, '/');
  const pad = b64.length % 4 ? '='.repeat(4 - (b64.length % 4)) : '';
  const bin = atob(b64 + pad);
  const out = new Uint8Array(bin.length);
  for (let i = 0; i < bin.length; i++) out[i] = bin.charCodeAt(i);
  return out;
};
```

FILE: packages/shared/src/permissions/keyPinning.ts

```
import type { SigningPublicKey } from './receiptSignature';

/**
 * Trust-on-first-use pinning for the org's receipt signing key. On first sight we
 * PIN { keyId -> publicKey }. Later, the same keyId MUST present the same public
 * key, or it's a substitution (fail closed). Rotation introduces a NEW keyId,
 * which is pinned fresh — a legitimate rotation never reuses a keyId with a
 * different key.
 */
```

```

export interface PinnedKey {
  keyId: string;
  publicKey: string;
  pinnedAt: string;
}

export type PinResult =
  | { status: 'pinned_new' }           // first time we saw this keyId
  | { status: 'match' }               // keyId + publicKey match the pin
  | { status: 'mismatch' };           // SAME keyId, DIFFERENT key => substitution

/**
 * Evaluate an incoming public key against the existing pins. Pure: returns the
 * result + the (possibly updated) pin set so the caller can persist it.
 */
export function evaluatePin(
  pins: Record<string, PinnedKey>,
  incoming: SigningPublicKey,
  now: Date = new Date(),
): { result: PinResult; pins: Record<string, PinnedKey> } {
  const existing = pins[incoming.keyId];
  if (!existing) {
    return {
      result: { status: 'pinned_new' },
      pins: { ...pins, [incoming.keyId]: { keyId: incoming.keyId, publicKey: incoming.publicKey, pinnedAt:
now.toISOString() } },
    };
  }
  if (existing.publicKey === incoming.publicKey) {
    return { result: { status: 'match' }, pins };
  }
  return { result: { status: 'mismatch' }, pins }; // do NOT overwrite a mismatching pin
}

/** A pin result is acceptable for trusting a signature only if new or matching. */
export function isPinAcceptable(result: PinResult): boolean {
  return result.status === 'pinned_new' || result.status === 'match';
}

```

FILE: packages/shared/src/permissions/index.ts [MODIFY]

```

// ...existing exports through Pack 27...
export * from './webcryptoEd25519'; // ADD
export * from './keyPinning';        // ADD

```

Part B — Desktop: signing key pin store

FILE: apps/desktop/src/renderer/src/services/signingKeyPinStore.ts

```

import {
  evaluatePin,
  isPinAcceptable,
  type PinResult,
  type PinnedKey,
  type SigningPublicKey,

```

```

} from '@remotedesk/shared';

/**
 * Persists pinned signing public keys in localStorage and evaluates incoming keys
 * against them. Fail-closed: a same-keyId/different-key mismatch is never
 * auto-accepted and never overwrites the existing pin. Rotation (new keyId) pins
 * fresh automatically.
 */
const PIN_KEY = 'remotedesk.signingKeyPins';

export class SigningKeyPinStore {
  private load(): Record<string, PinnedKey> {
    try {
      return JSON.parse(localStorage.getItem(PIN_KEY) ?? '{}') as Record<string, PinnedKey>;
    } catch {
      return {};
    }
  }

  private save(pins: Record<string, PinnedKey>): void {
    try {
      localStorage.setItem(PIN_KEY, JSON.stringify(pins));
    } catch {
      /* storage unavailable: pins are best-effort this session */
    }
  }

  /** Evaluate + persist (only on new/match; a mismatch leaves the pin untouched). */
  check(incoming: SigningPublicKey): PinResult {
    const pins = this.load();
    const { result, pins: next } = evaluatePin(pins, incoming);
    if (isPinAcceptable(result)) this.save(next);
    return result;
  }

  /** Get the pinned public key for a keyId, if any. */
  pinned(keyId: string): PinnedKey | null {
    return this.load()[keyId] ?? null;
  }

  /** Operator escape hatch: forget a pin (e.g. after a verified rotation incident). */
  forget(keyId: string): void {
    const pins = this.load();
    delete pins[keyId];
    this.save(pins);
  }
}

```

Part C — Desktop: receipt verify client

FILE: apps/desktop/src/renderer/src/services/receiptVerifyClient.ts

```

import {
  verifyReceiptSignature,
  verifyReceipt,

```

```

webcryptoEd25519Verify,
isPinAcceptable,
type ConsentReceipt,
type SigningPublicKey,
type PinResult,
} from '@remotedesk/shared';
import { SigningKeyPinStore } from '../signingKeyPinStore';

/**
 * Full desktop-side verification of a signed consent receipt:
 * 1. hash-chain intact (Pack 22 verifyReceipt, with WebCrypto sha256)
 * 2. signing key pin acceptable (Pack 28 pinning) — detects key substitution
 * 3. Ed25519 signature valid over the canonical preimage (Pack 26 verifier)
 * All three must pass. Fail-closed: any failure => verified:false with a reason.
 * No server call, no private material.
 */
export interface DesktopVerifyResult {
  verified: boolean;
  chainValid: boolean;
  signatureValid: boolean;
  pin: PinResult | 'no_signature';
  reason: string | null;
}

export class ReceiptVerifyClient {
  constructor(private readonly pins = new SigningKeyPinStore()) {}

  async verify(orgId: string, receipt: ConsentReceipt, publicKey: SigningPublicKey | null): Promise<DesktopVerifyResult>
  {
    // 1. Chain integrity (no signature needed).
    const chain = await verifyReceipt(receipt, sha256Hex);
    if (!chain.valid) {
      return { verified: false, chainValid: false, signatureValid: false, pin: 'no_signature', reason: 'chain_invalid' };
    }

    // No signature on the receipt yet (server didn't sign) => chain-only, unverified sig.
    if (!receipt.signature || !publicKey) {
      return { verified: false, chainValid: true, signatureValid: false, pin: 'no_signature', reason: 'unsigned' };
    }

    // 2. Pin check (TOFU). Mismatch => possible key substitution.
    const pin = this.pins.check(publicKey);
    if (!isPinAcceptable(pin)) {
      return { verified: false, chainValid: true, signatureValid: false, pin, reason: 'key_pin_mismatch' };
    }

    // 3. Signature check against the (pinned) public key.
    const signatureValid = await verifyReceiptSignature(orgId, receipt, receipt.signature, publicKey,
webcryptoEd25519Verify);
    return {
      verified: signatureValid,
      chainValid: true,
      signatureValid,
      pin,
      reason: signatureValid ? null : 'signature_invalid',
    };
  }
}

```

```

}

async function sha256Hex(input: string): Promise<string> {
  const digest = await crypto.subtle.digest('SHA-256', new TextEncoder().encode(input));
  return Array.from(new Uint8Array(digest), (b) => b.toString(16).padStart(2, '0')).join('');
}

```

Part D — Desktop: wiring + UI

FILE: apps/desktop/src/renderer/src/services/consentReceiptService.ts [MODIFY]

```

// ...existing Pack 22 imports...
import { ReceiptVerifyClient, type DesktopVerifyResult } from './receiptVerifyClient';
import type { SigningPublicKey } from '@remotedesk/shared';

// ---- ADD: a verifier instance ----
// private verifier = new ReceiptVerifyClient();

// ---- ADD: fetch the org public key + verify a (server-signed) receipt ----
// async verifySigned(orgId: string, receipt: ConsentReceipt, fetchPublicKey: () => Promise<SigningPublicKey | null>):
// Promise<DesktopVerifyResult> {
//   const pub = receipt.signature ? await fetchPublicKey().catch(() => null): null;
//   return this.verifier.verify(orgId, receipt, pub);
// }
//
// fetchPublicKey() should call GET /api/devices/receipts/signing-key (Pack 26).
// The desktop verifies the SIGNATURE offline; it only fetches the PUBLIC key,
// which is pinned on first use so a later swap is caught.

```

FILE: apps/desktop/src/renderer/src/components/ReceiptVerificationBadge.tsx

```

import React from 'react';
import type { DesktopVerifyResult } from '../services/receiptVerifyClient';

/**
 * Badge summarizing the desktop-side verification of a signed receipt. Makes the
 * key-substitution case loud (it's a security signal, not a normal failure).
 */
export function ReceiptVerificationBadge({ result }: { result: DesktopVerifyResult | null }) {
  if (!result) return <span className="rv-badge rv-badge--pending">not verified yet</span>;
  if (result.verified) return <span className="rv-badge rv-badge--ok">✓ verified (signature + chain)</span>;

  switch (result.reason) {
    case 'unsigned':
      return <span className="rv-badge rv-badge--warn">chain intact · not signed yet</span>;
    case 'chain_invalid':
      return <span className="rv-badge rv-badge--bad">⊠ chain tampered</span>;
    case 'key_pin_mismatch':
      return <span className="rv-badge rv-badge--danger">⚠ signing key changed — possible substitution</span>;
    case 'signature_invalid':
      return <span className="rv-badge rv-badge--bad">⊠ signature invalid</span>;
    default:
      return <span className="rv-badge rv-badge--bad">⊠ unverified</span>;
  }
}

```

```
}  
}
```

FILE: apps/desktop/src/renderer/src/components/SessionSummaryDialog.tsx [MODIFY]

```
// ...existing Pack 22 imports...  
import { ReceiptVerificationBadge } from './ReceiptVerificationBadge';  
import type { DesktopVerifyResult } from '../services/receiptVerifyClient';  
  
// ---- ADD: state + a verify action that runs the desktop verifier ----  
// const [verifyResult, setVerifyResult] = useState<DesktopVerifyResult | null>(null);  
// const runVerify = async () => {  
//   if (!receipt) return;  
//   const r = await service.verifySigned(orgId, receipt, fetchSigningPublicKey);  
//   setVerifyResult(r);  
// };  
//  
// fetchSigningPublicKey = () => GET /api/devices/receipts/signing-key (Pack 26).  
  
// ---- ADD to the dialog actions: a Verify button + the badge ----  
// <button type="button" className="btn btn--ghost" onClick={runVerify}>Verify signature</button>  
// <ReceiptVerificationBadge result={verifyResult} />  
//  
// The existing Pack 22 download + chain-only verify stay as-is; this adds the  
// independent, signature-level confirmation the host can trust on its own.
```

Part E — Tests

FILE: tests/permissions/webcryptoEd25519.test.ts

```
import { describe, it, expect, beforeAll } from 'vitest';  
import { webcrypto } from 'node:crypto';  
import { webcryptoEd25519Verify } from '@remotedesk/shared';  
import { generateEd25519, signEd25519 } from '../../apps/api/src/lib/ed25519';  
  
// Provide WebCrypto in the Node test env (renderer uses crypto.subtle).  
beforeAll(() => {  
  if (!(globalThis as any).crypto) (globalThis as any).crypto = webcrypto as any;  
  if (!(globalThis as any).atob) (globalThis as any).atob = (s: string) => Buffer.from(s, 'base64').toString('binary');  
});  
  
describe('webcryptoEd25519Verify', () => {  
  it('verifies a signature produced by the node signer (cross-impl)', async () => {  
    const { publicKey, privateKey } = generateEd25519();  
    const msg = 'remotedesk-receipt-v1|org|r1|hash|2026-06-15T10:00:00.000Z';  
    const sig = signEd25519(privateKey, msg);  
    expect(await webcryptoEd25519Verify(publicKey, msg, sig)).toBe(true);  
  });  
  
  it('rejects a tampered message', async () => {  
    const { publicKey, privateKey } = generateEd25519();  
    const sig = signEd25519(privateKey, 'original');  
    expect(await webcryptoEd25519Verify(publicKey, 'tampered', sig)).toBe(false);  
  });  
});
```

```

    it('fails closed on a bad key (no throw)', async () => {
      expect(await webcryptoEd25519Verify('!!notbase64!!', 'm', 'sig')).toBe(false);
    });
  });
});

```

FILE: tests/permissions/keyPinning.test.ts

```

import { describe, it, expect } from 'vitest';
import { evaluatePin, isPinAcceptable, type PinnedKey, type SigningPublicKey } from '@remotedesk/shared';

function pub(keyId: string, publicKey: string): SigningPublicKey {
  return { keyId, publicKey, alg: 'ed25519', createdAt: 'now', retiredAt: null };
}

describe('keyPinning', () => {
  it('pins a new keyId on first sight', () => {
    const { result, pins } = evaluatePin({}, pub('k1', 'PUB1'));
    expect(result.status).toBe('pinned_new');
    expect(pins.k1.publicKey).toBe('PUB1');
    expect(isPinAcceptable(result)).toBe(true);
  });

  it('matches when the same keyId presents the same key', () => {
    const pins: Record<string, PinnedKey> = { k1: { keyId: 'k1', publicKey: 'PUB1', pinnedAt: 'now' } };
    const { result } = evaluatePin(pins, pub('k1', 'PUB1'));
    expect(result.status).toBe('match');
    expect(isPinAcceptable(result)).toBe(true);
  });

  it('flags a mismatch when the same keyId presents a different key', () => {
    const pins: Record<string, PinnedKey> = { k1: { keyId: 'k1', publicKey: 'PUB1', pinnedAt: 'now' } };
    const { result, pins: next } = evaluatePin(pins, pub('k1', 'EVIL'));
    expect(result.status).toBe('mismatch');
    expect(isPinAcceptable(result)).toBe(false);
    expect(next.k1.publicKey).toBe('PUB1'); // pin NOT overwritten
  });

  it('pins a rotated key (new keyId) fresh', () => {
    const pins: Record<string, PinnedKey> = { k1: { keyId: 'k1', publicKey: 'PUB1', pinnedAt: 'now' } };
    const { result, pins: next } = evaluatePin(pins, pub('k2', 'PUB2'));
    expect(result.status).toBe('pinned_new');
    expect(next.k2.publicKey).toBe('PUB2');
    expect(next.k1.publicKey).toBe('PUB1'); // old pin retained for old receipts
  });
});

```

FILE: tests/permissions/receiptVerifyClient.test.ts

```

import { describe, it, expect, beforeAll, vi } from 'vitest';
import { webcrypto } from 'node:crypto';
import { buildChain, signingPreimage, makeAuditEvent, type ConsentReceipt, type SessionSummary, type SigningPublicKey }
from '@remotedesk/shared';
import { generateEd25519, signEd25519 } from '../../apps/api/src/lib/ed25519';
import { ReceiptVerifyClient } from '../../apps/desktop/src/renderer/src/services/receiptVerifyClient';

beforeAll(() => {

```

```

    if (!(globalThis as any).crypto) (globalThis as any).crypto = webcrypto as any;
    if (!(globalThis as any).atob) (globalThis as any).atob = (s: string) => Buffer.from(s, 'base64').toString('binary');
    // in-memory localStorage shim for SigningKeyPinStore
    const mem: Record<string, string> = {};
    (globalThis as any).localStorage = {
      getItem: (k: string) => mem[k] ?? null,
      setItem: (k: string, v: string) => { mem[k] = v; },
      removeItem: (k: string) => { delete mem[k]; },
    };
  });

const sha256Hex = async (i: string) => {
  const d = await webcrypto.subtle.digest('SHA-256', new TextEncoder().encode(i));
  return Array.from(new Uint8Array(d), (b) => b.toString(16).padStart(2, '0')).join('');
};

const summary: SessionSummary = {
  deviceId: 'd1', startedAt: '2026-06-15T10:00:00.000Z', endedAt: '2026-06-15T10:01:00.000Z',
  durationSeconds: 60, capabilitiesUsed: { remoteInput: false, clipboardSync: false, fileTransfer: false },
  unattended: false, counts: { inputBlocked: 0, inputRateLimited: 0, emergencyStops: 0, controlGrants: 1, controlRevokes: 0 },
  endReason: 'user_disconnect',
};

async function signedReceipt(orgId: string, kp: { publicKey: string; privateKey: string }, keyId = 'k1'): Promise<{
  receipt: ConsentReceipt; pub: SigningPublicKey }> {
  const { lineItems, rootHash } = await buildChain([makeAuditEvent('control.granted')], sha256Hex);
  const base: ConsentReceipt = { receiptId: 'r1', summary, lineItems, rootHash, issuedAt: '2026-06-15T10:01:01.000Z',
  version: 1 };
  const sig = signEd25519(kp.privateKey, signingPreimage(orgId, base));
  const receipt: ConsentReceipt = { ...base, signature: { keyId, signature: sig, alg: 'ed25519', signedAt: 'now' } };
  return { receipt, pub: { keyId, publicKey: kp.publicKey, alg: 'ed25519', createdAt: 'now', retiredAt: null } };
}

describe('ReceiptVerifyClient', () => {
  it('verifies a properly signed receipt (chain + pin + signature)', async () => {
    const kp = generateEd25519();
    const { receipt, pub } = await signedReceipt('org1', kp);
    const client = new ReceiptVerifyClient();
    const r = await client.verify('org1', receipt, pub);
    expect(r.verified).toBe(true);
    expect(r.chainValid).toBe(true);
    expect(r.signatureValid).toBe(true);
  });

  it('reports unsigned when there is no signature', async () => {
    const kp = generateEd25519();
    const { receipt } = await signedReceipt('org1', kp);
    const unsigned = { ...receipt, signature: undefined };
    const r = await new ReceiptVerifyClient().verify('org1', unsigned, null);
    expect(r.reason).toBe('unsigned');
    expect(r.chainValid).toBe(true);
  });

  it('flags key substitution: same keyId, different public key', async () => {
    const good = generateEd25519();
    const evil = generateEd25519();
  });

```

```

const client = new ReceiptVerifyClient();
const a = await signedReceipt('org1', good, 'k1');
await client.verify('org1', a.receipt, a.pub); // pins k1 -> good key
// now an attacker presents k1 with a different public key
const b = await signedReceipt('org1', evil, 'k1');
const r = await client.verify('org1', b.receipt, b.pub);
expect(r.reason).toBe('key_pin_mismatch');
expect(r.verified).toBe(false);
});

it('detects a tampered chain regardless of signature', async () => {
  const kp = generateEd25519();
  const { receipt, pub } = await signedReceipt('org1', kp);
  receipt.lineItems[0] = { ...receipt.lineItems[0], type: 'emergency_stop.engaged' };
  const r = await new ReceiptVerifyClient().verify('org1', receipt, pub);
  expect(r.chainValid).toBe(false);
  expect(r.verified).toBe(false);
});
});

```

Part F — Reports

FILE: [generated-pack28-desktop-receipt-verify-merge-summary.md](#)

Pack 28 (Desktop) — Receipt Signature Verification + Key Pinning · Merge Summary

What this slice does

Closes the verification loop on the desktop. After a session ends and the server returns a signed receipt (Pack 26), the host independently:

1. re-checks the Pack 22 hash chain (WebCrypto SHA-256),
2. pins the org's signing public key on first use (TOFU) + detects substitution,
3. verifies the Ed25519 signature offline (Pack 26 shared verifier via WebCrypto).

All three must pass; otherwise the badge shows exactly why (incl. a loud key-substitution warning). No server trust required beyond fetching the public key.

Files created (8)

- packages/shared/src/permissions/webcryptoEd25519.ts
- packages/shared/src/permissions/keyPinning.ts
- apps/desktop/src/renderer/src/services/signingKeyPinStore.ts
- apps/desktop/src/renderer/src/services/receiptVerifyClient.ts
- apps/desktop/src/renderer/src/components/ReceiptVerificationBadge.tsx
- tests/permissions/webcryptoEd25519.test.ts
- tests/permissions/keyPinning.test.ts
- tests/permissions/receiptVerifyClient.test.ts

Files modified (3)

- packages/shared/src/permissions/index.ts — export webcryptoEd25519 + keyPinning.
- apps/desktop/src/renderer/src/services/consentReceiptService.ts — verifySigned(orgId, receipt, fetchPublicKey).
- apps/desktop/src/renderer/src/components/SessionSummaryDialog.tsx — Verify button + badge.

Files intentionally NOT touched

- desktop main/preload — no native, no bridge.
- apps/api/** — signing + public-key publishing already exist (26/27).
- receiptSignature.ts — Pack 26 contract reused unchanged.

```
## Remaining manual wiring risks
1. fetchSigningPublicKey -> GET /api/devices/receipts/signing-key (Pack 26).
2. Ed25519 in WebCrypto: ensure your Electron/Chromium build supports it (recent versions do); otherwise polyfill — the verifier already fails closed if absent.
3. Decide the UX for a pin mismatch (block download? warn loudly?). Default: badge shows danger.
4. localStorage is the pin store; clearing it re-pins (TOFU) — acceptable, like SSH known_hosts.

## Verify
- pnpm --filter @remotedesk/shared build && pnpm -r test
- Manual: end a session -> Verify signature -> "verified"; simulate a swapped key (same keyId, different pubkey) -> "signing key changed" warning.
```

FILE: [generated-pack28-desktop-receipt-verify-code-review.md](#)

```
# Pack 28 (Desktop) — Code Review Notes

## Strengths
- True client-side verification: chain + pin + Ed25519 signature, all offline; the host doesn't take the server's word.
- Cross-impl correctness: a node-signed signature verifies under WebCrypto (test asserts it).
- TOFU pinning catches signing-key substitution (same keyId, different key) and never overwrites a mismatching pin.
- Rotation-safe: a new keyId pins fresh; old keyIds keep their pins for old receipts.
- Everything fails closed: bad key, unsupported crypto, mismatch, tamper => verified:false with a reason.

## Reviewer checklist
- [ ] WebCrypto Ed25519 supported in the target Electron/Chromium; else polyfill.
- [ ] fetchSigningPublicKey points at the Pack 26 public-key route.
- [ ] Pin store persistence acceptable (localStorage); consider main-process secure store for stronger pinning.
- [ ] Pin-mismatch UX surfaces prominently (security signal).
- [ ] No private material ever requested/handled on the desktop (verified: public key only).

## Safety
- Desktop never holds a private key; verifies with the published public key only.
- Key substitution is detected on the client, not trusted from the server.
- Content-free: verification reads ids/hashes/signature, never session content.
```

FILE: [generated-pack28-desktop-receipt-verify-manifest.json](#)

```
{
  "pack": "pack28-desktop-receipt-verify",
  "title": "Desktop-Side Receipt Signature Verification + Public-Key Pinning",
  "actualFileCount": 11,
  "filesCreated": [
    "packages/shared/src/permissions/webcryptoEd25519.ts",
    "packages/shared/src/permissions/keyPinning.ts",
    "apps/desktop/src/renderer/src/services/signingKeyPinStore.ts",
    "apps/desktop/src/renderer/src/services/receiptVerifyClient.ts",
    "apps/desktop/src/renderer/src/components/ReceiptVerificationBadge.tsx",
    "tests/permissions/webcryptoEd25519.test.ts",
    "tests/permissions/keyPinning.test.ts",
    "tests/permissions/receiptVerifyClient.test.ts"
  ],
  "filesModified": [
    "packages/shared/src/permissions/index.ts",
    "apps/desktop/src/renderer/src/services/consentReceiptService.ts",
    "apps/desktop/src/renderer/src/components/SessionSummaryDialog.tsx"
  ],
}
```

```

"filesNotTouched": [
  "apps/desktop/src/main/index.ts",
  "apps/desktop/src/preload/index.ts",
  "apps/api/**",
  "packages/shared/src/permissions/receiptSignature.ts"
],
"dependenciesRequired": {
  "new": [],
  "existing": ["@remotedesk/shared", "react", "vitest", "WebCrypto (crypto.subtle Ed25519)"],
  "assumedFromProject": [
    "Pack 22 consent receipt + verifyReceipt",
    "Pack 26 receiptSignature verifier + GET signing-key route",
    "Pack 27 rotatable keys (new keyId per rotation)",
    "renderer localStorage + WebCrypto"
  ]
},
"knownLimitations": [
  "WebCrypto Ed25519 requires a recent Chromium/Electron; verifier fails closed if unavailable (consider a polyfill).",
  "Pin store uses localStorage (TOFU like SSH known_hosts); a main-process secure store would strengthen it.",
  "Desktop fetches the public key over the network; pinning catches later substitution but first-use trusts the first key seen.",
  "Verification is per-receipt on demand; no background batch re-verify."
],
"remainingManualWiringRisks": [
  "Wire fetchSigningPublicKey to the Pack 26 route.",
  "Confirm WebCrypto Ed25519 support in the target build.",
  "Decide pin-mismatch UX (warn vs block).",
],
"estimatedCompletionAfterCarefulMerge": "86-90%"
}

```

Pack 28 done: 11 files (8 created, 3 modified). The desktop can now verify its own consent receipts end to end without trusting the server's word: it re-checks the Pack 22 hash chain, pins the org's signing public key on first use, and verifies the Ed25519 signature offline with WebCrypto. All three must pass, and a same-keyId/different-key mismatch raises a loud "signing key changed, possible substitution" warning instead of silently accepting.

The clean bit: the cross-implementation correctness is guaranteed by reuse, the WebCrypto verifier feeds the SAME shared `verifyReceiptSignature` the server uses, and a test proves a node-signed signature verifies under WebCrypto, so the two implementations can't drift. Pinning is rotation-safe by construction: a new keyId (legit rotation from Pack 27) pins fresh while old keyIds keep their pins for old receipts, and a mismatching pin is never overwritten. The desktop never touches private material, just the published public key.

That's the receipt trust chain fully closed on both ends: server signs + protects + rotates (26/27), client independently verifies + pins (28). Next slice = scheduled/auto key rotation with overlap windows, or a public "verify this receipt" web page for third parties (auditors) using the same shared verifier. Bolo.